

Skip – List
–Delete Element
–Program Analysis – Type A

Program Analysis:

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
```

```
        lst->level--;  
    }  
    cout << "Successfully deleted key " << key << endl;  
} else {  
    cout << "Key " << key << " not found. Cannot delete." << endl;  
}  
}  
  
... .  
case 2:  
    cout << "Enter key to delete: ";  
    cin >> key;  
    deleteElement(list, key);  
    break;
```

Program Analysis

```
deleteElement(list, key);
```

deleteElement(list: ff08, key: 30) → passed .

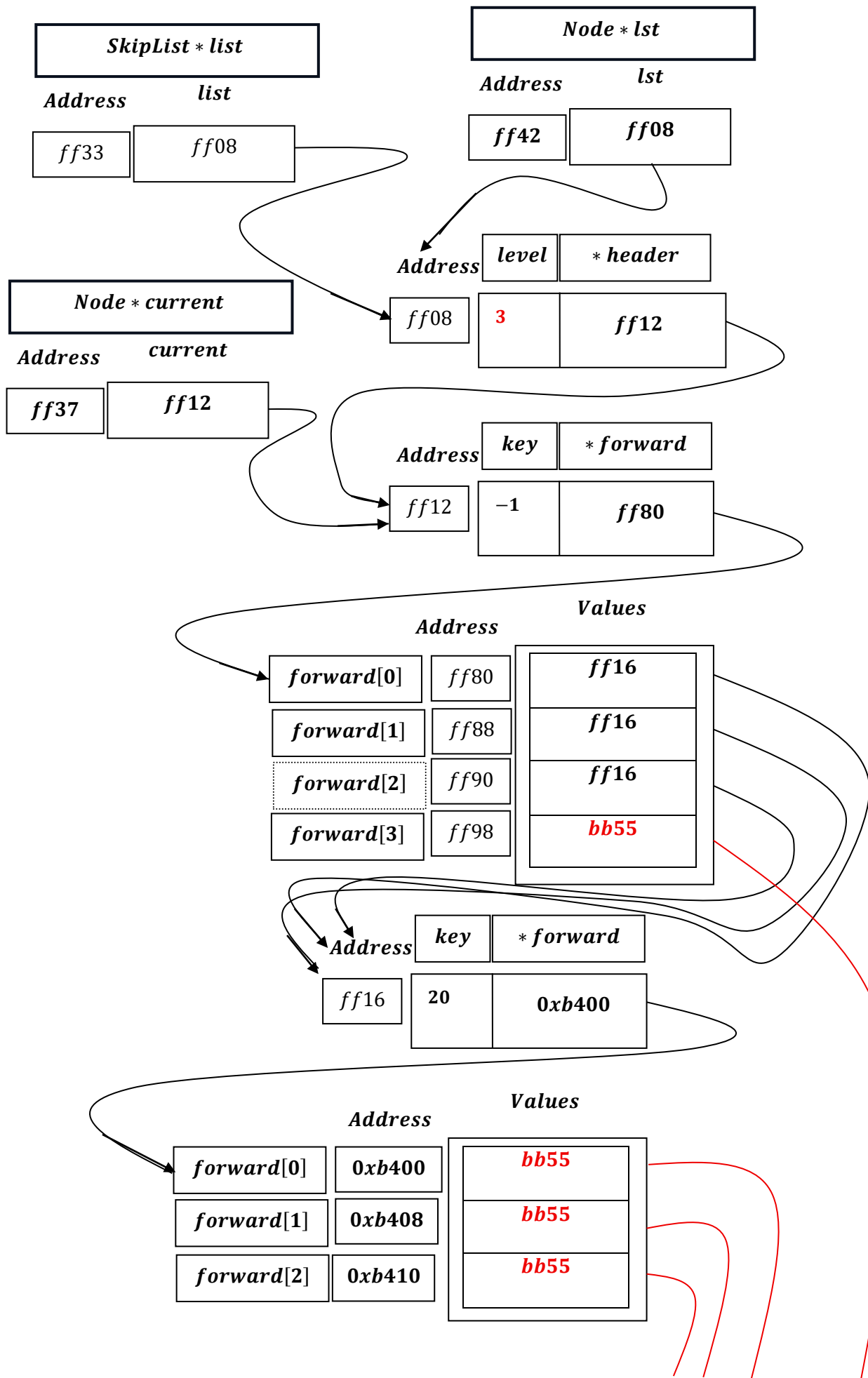
And,

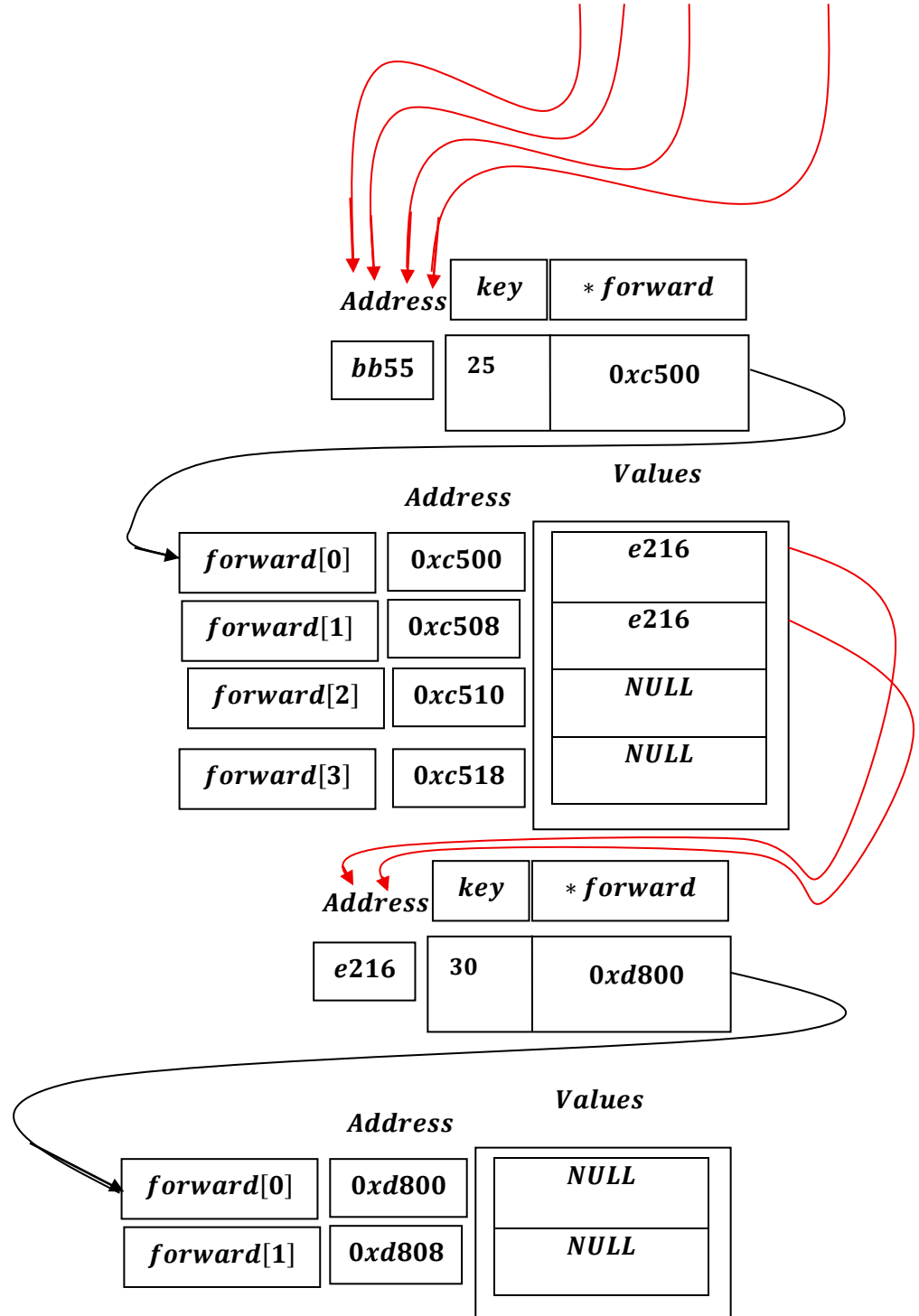
```
void deleteElement(struct SkipList *lst, int key)
```

deleteElement(list := lst: ff08, key := key: 15) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.





```
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is *update*[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

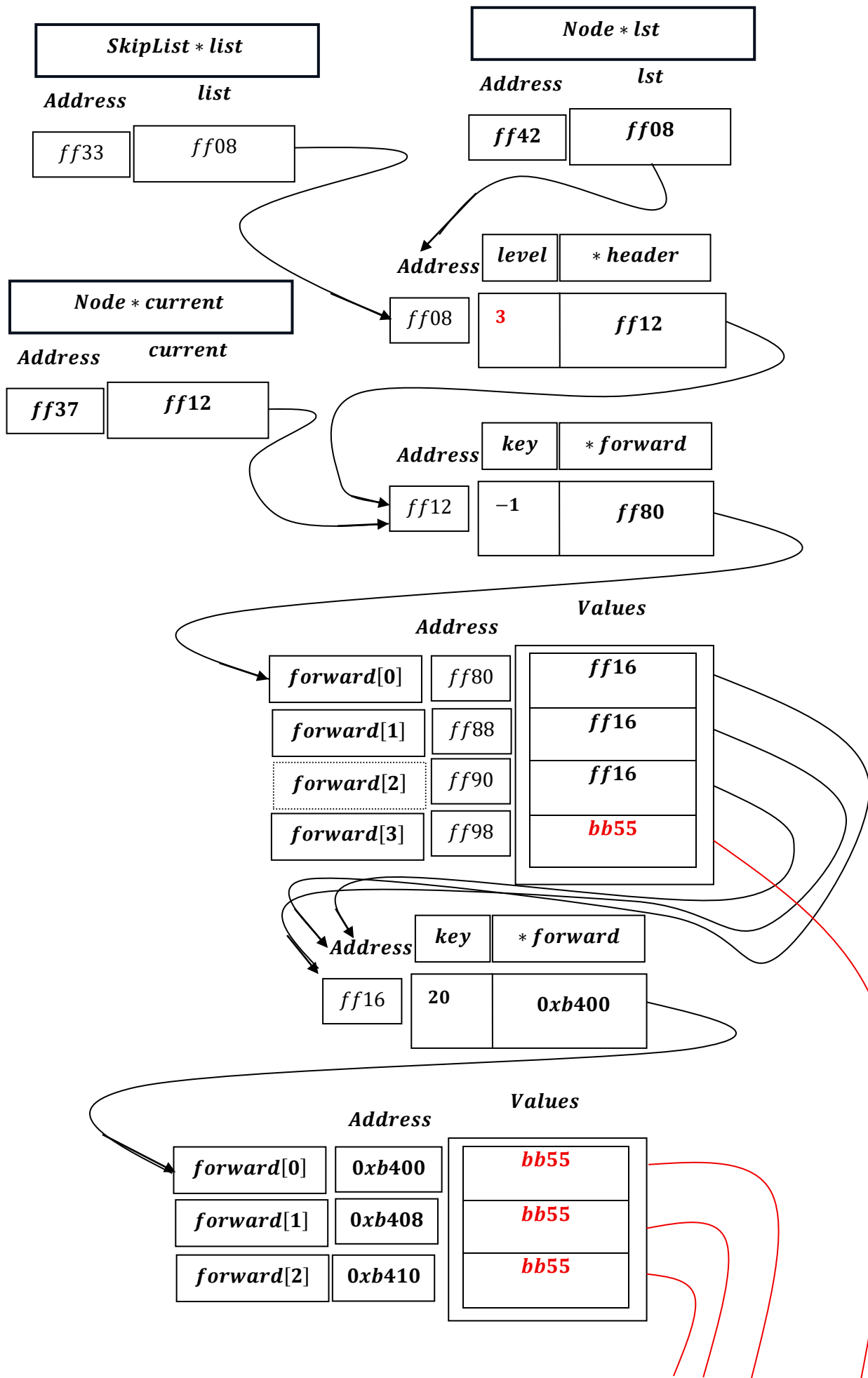
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

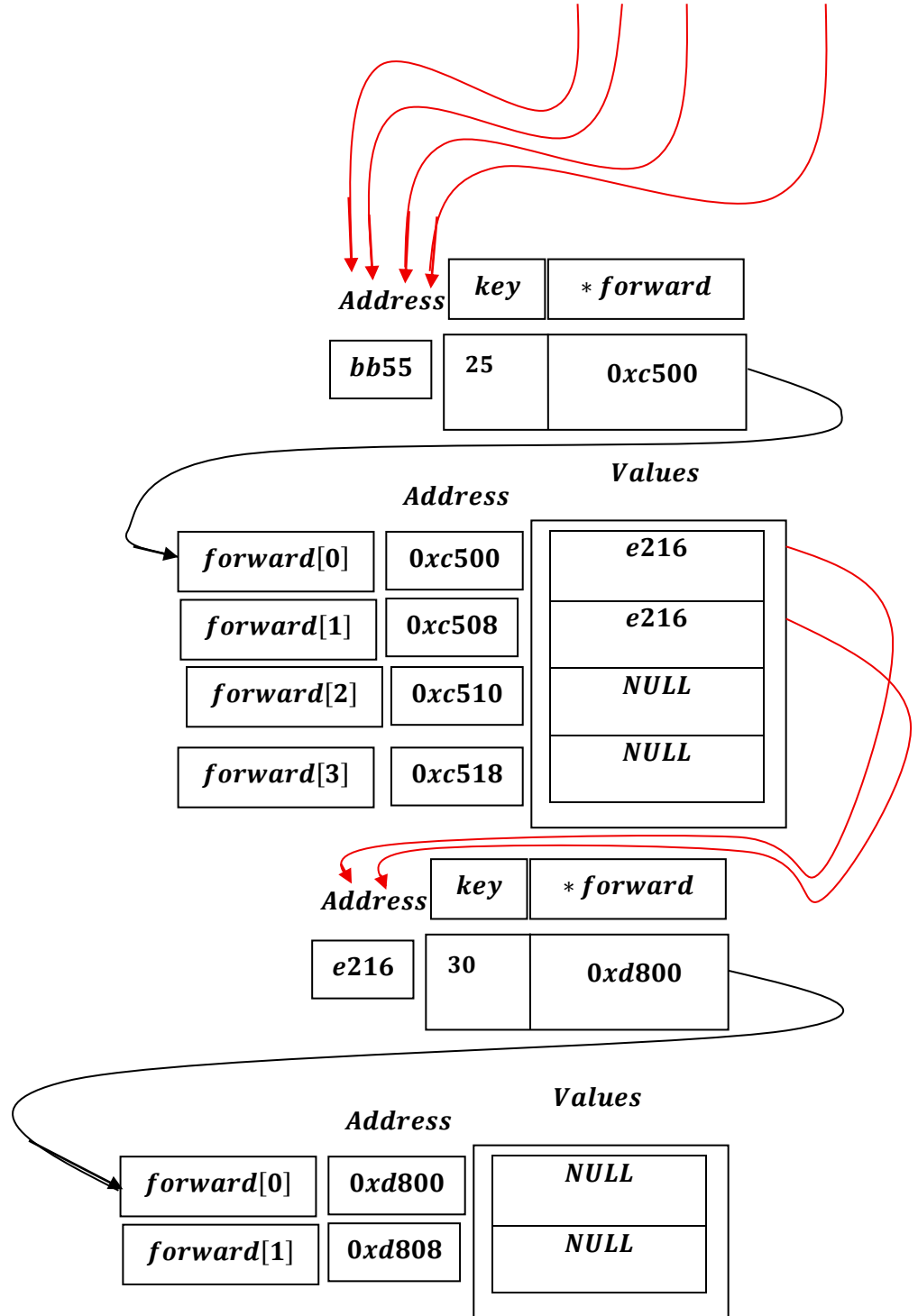
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>NULL</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>NULL</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

*Here we kept addresses same like: 0x7fff0000
, just for an example , but it may not be same ,
in reality.*





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

$i = lst \rightarrow level = 3; i \geq 0 :$

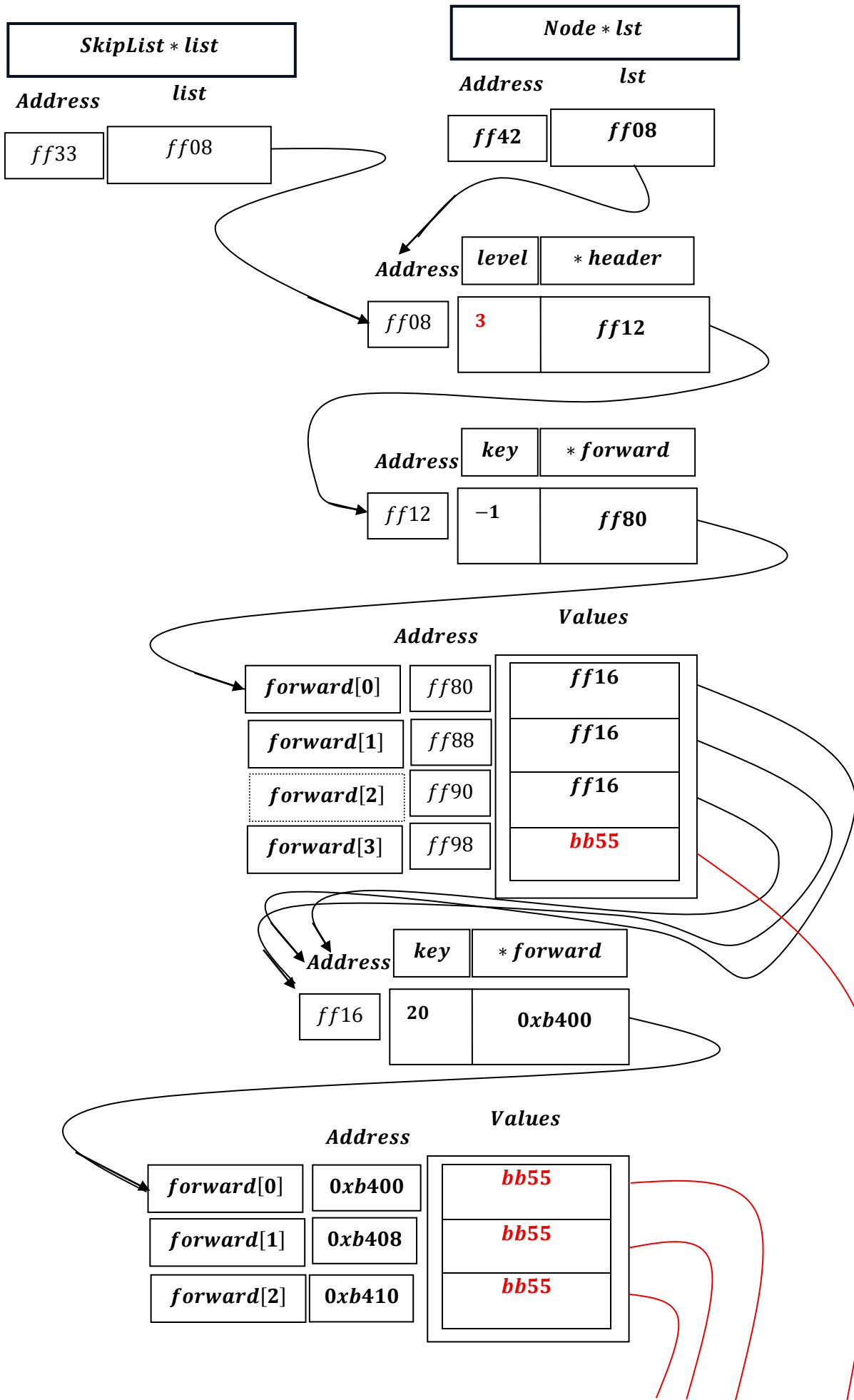
$current: ff12 \rightarrow forward[3]: ff98 \rightarrow bb55 \neq NULL[True] \text{ AND}$

$current: ff12 \rightarrow forward[3]: bb55 \rightarrow key: 25 < key: 30[True]$

$current = current: ff12 \rightarrow forward[3]: bb55$

$current: bb55 \rightarrow forward[3]: 0xc518 \rightarrow NULL \neq NULL[False]$

$update[3] = current: bb55.$



*Node * current*

Address

current

ff37

bb55

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

NULL

update[1]

0x7fff0008

NULL

update[2]

0x7fff0010

NULL

update[3]

0x7fff0018

bb55

$$i = i - - = i - 1 = 3 - 1 = 2$$

$i = 2; i \geq 0 :$

$current: bb55 \rightarrow forward[2]: 0xc510 \rightarrow NULL \neq NULL[False]$

$update[2] = current: bb55.$

$$i = i - - = i - 1 = 2 - 1 = 1$$

$i = 1; i \geq 0 :$

$current: bb55 \rightarrow forward[1]: 0xc508 \rightarrow e216 \neq NULL[True] AND$

$current: bb55 \rightarrow forward[1]: e216 \rightarrow key: 30 < key: 30[False]$

$update[1] = current: bb55.$

$$i = i - - = i - 1 = 1 - 1 = 0$$

$i = 0; i \geq 0 :$

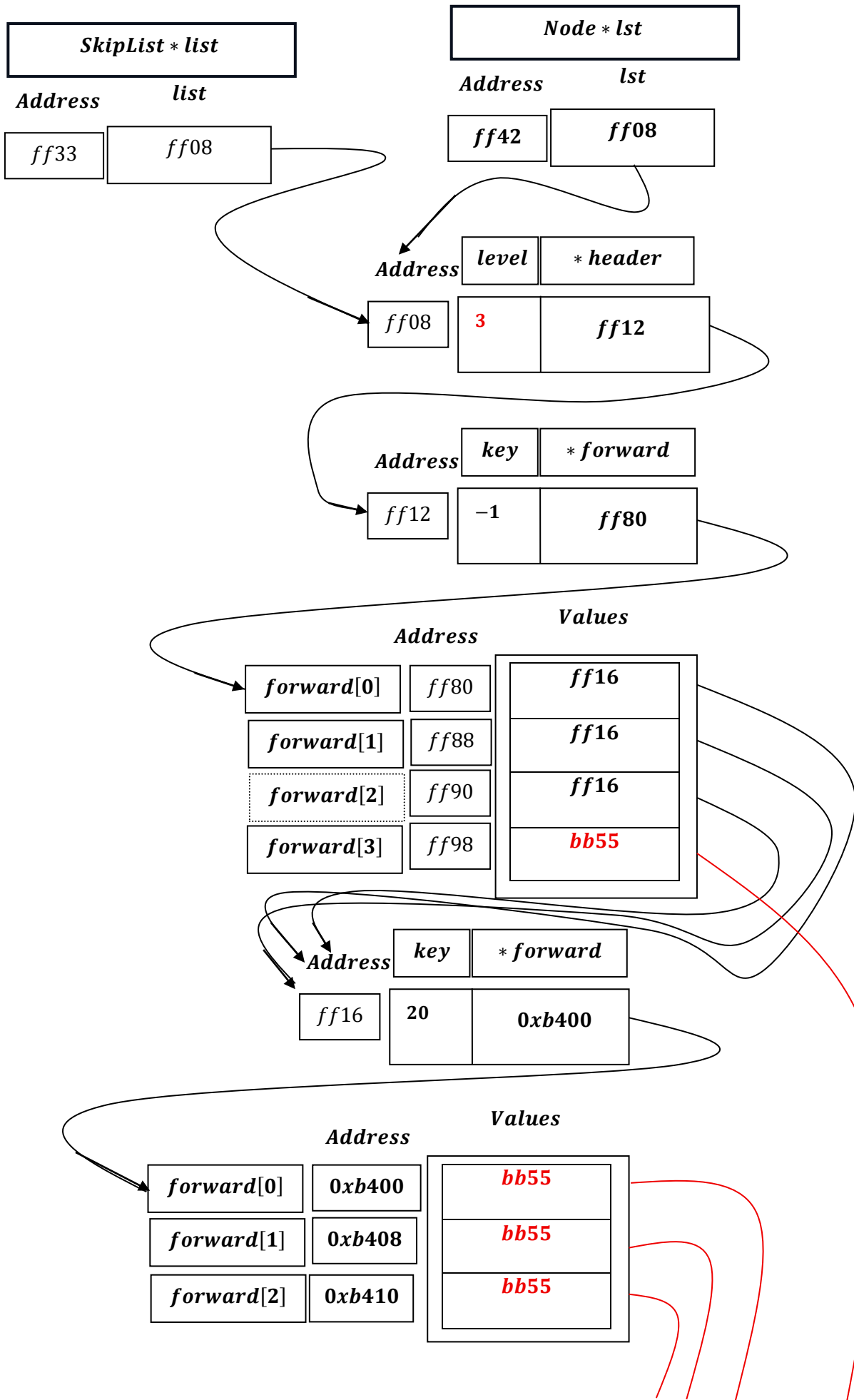
$current: bb55 \rightarrow forward[0]: 0xc500 \rightarrow e216 \neq NULL[True] AND$

$current: bb55 \rightarrow forward[0]: e216 \rightarrow key: 30 < key: 30[False]$

$update[0] = current: bb55.$

$$i = i - - = i - 1 = 0 - 1 = -1$$

$i = -1; i \geq 0 [Condition is False]$



*Node * current*

Address

current

ff37

bb55

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

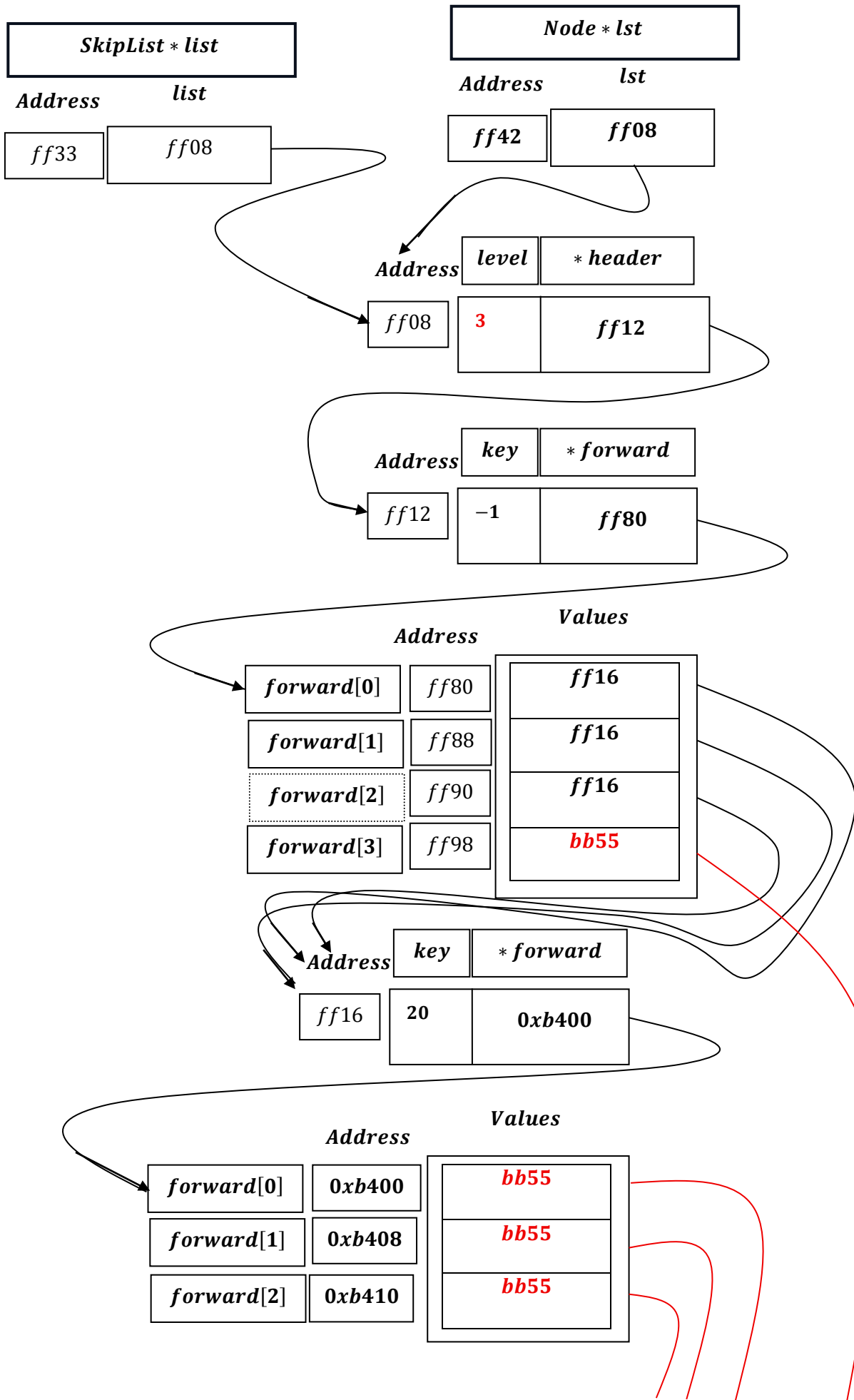
update[3]

0x7fff0018

bb55

```
current = current->forward[0];
```

current = *current*: *bb55* → *forward*[0]: *0xc500* → *e216*.



*Node * current*

Address *current*

<i>ff37</i>	<i>e216</i>
-------------	-------------

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55

```

if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
                // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
} else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}

```

if(current: e216 ≠ NULL is True and

current: e216 → key: 30 == 30 is True) :

for : int i = 0; i ≤ lst → level: 3 :

if(update[0]: bb55 → forward[0]: e216 ≠ current: e216) [false]

update[0]: bb55 → forward[0] = current: e216 → forward[0]
: NULL

i = i + 1 = 0 + 1 = 1.

for : int i = 1; i ≤ lst → level: 3 :

if(update[1]: bb55 → forward[1]: e216 ≠ current: e216) [false]

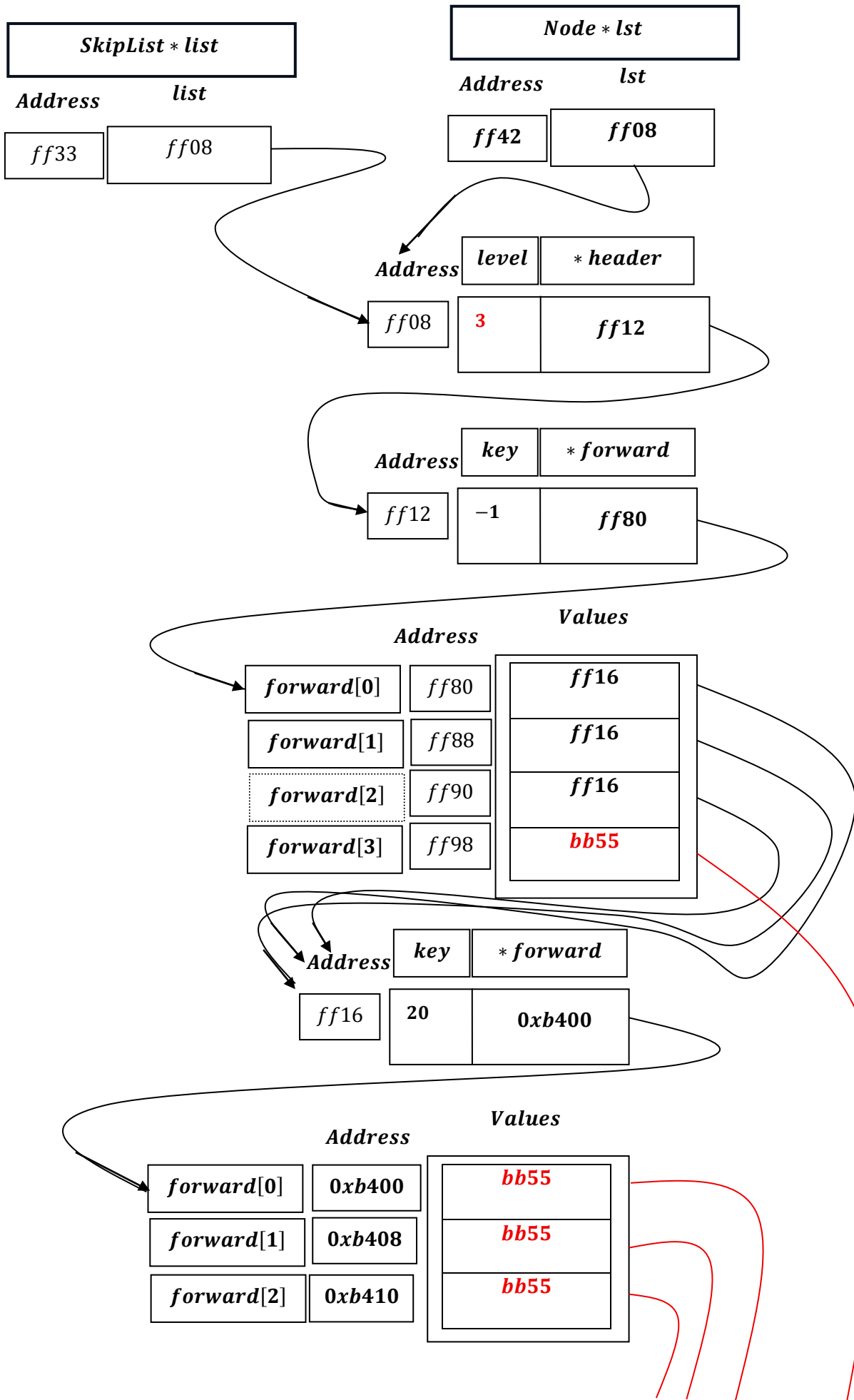
*update[1]: bb55 → forward[1] = current: e216 → forward[1]
: NULL*

i = i + 1 = 1 + 1 = 2.

for : int i = 2; i ≤ lst → level: 3 :

if(update[2]: bb55 → forward[2]: NULL ≠ current: e216) [True]

*break; The control flow statement break;
Terminates loop immediately terminates
here.*



*Node * current*

Address *current*

<i>ff37</i>	<i>e216</i>
-------------	-------------

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

NULL

forward[1]

0xc508

NULL

forward[2]

0xc510

NULL

forward[3]

0xc518

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

NULL

forward[1]

0xd808

NULL

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55

```
free(current->forward);  
free(current);
```

Now,

```
free(current->forward);
```

*The pointer **current** → **forward** holds the starting address of this chunk, which is **0xd800**. This is where **forward[0]** lives.*

***The Offset:** The address **0xd808** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

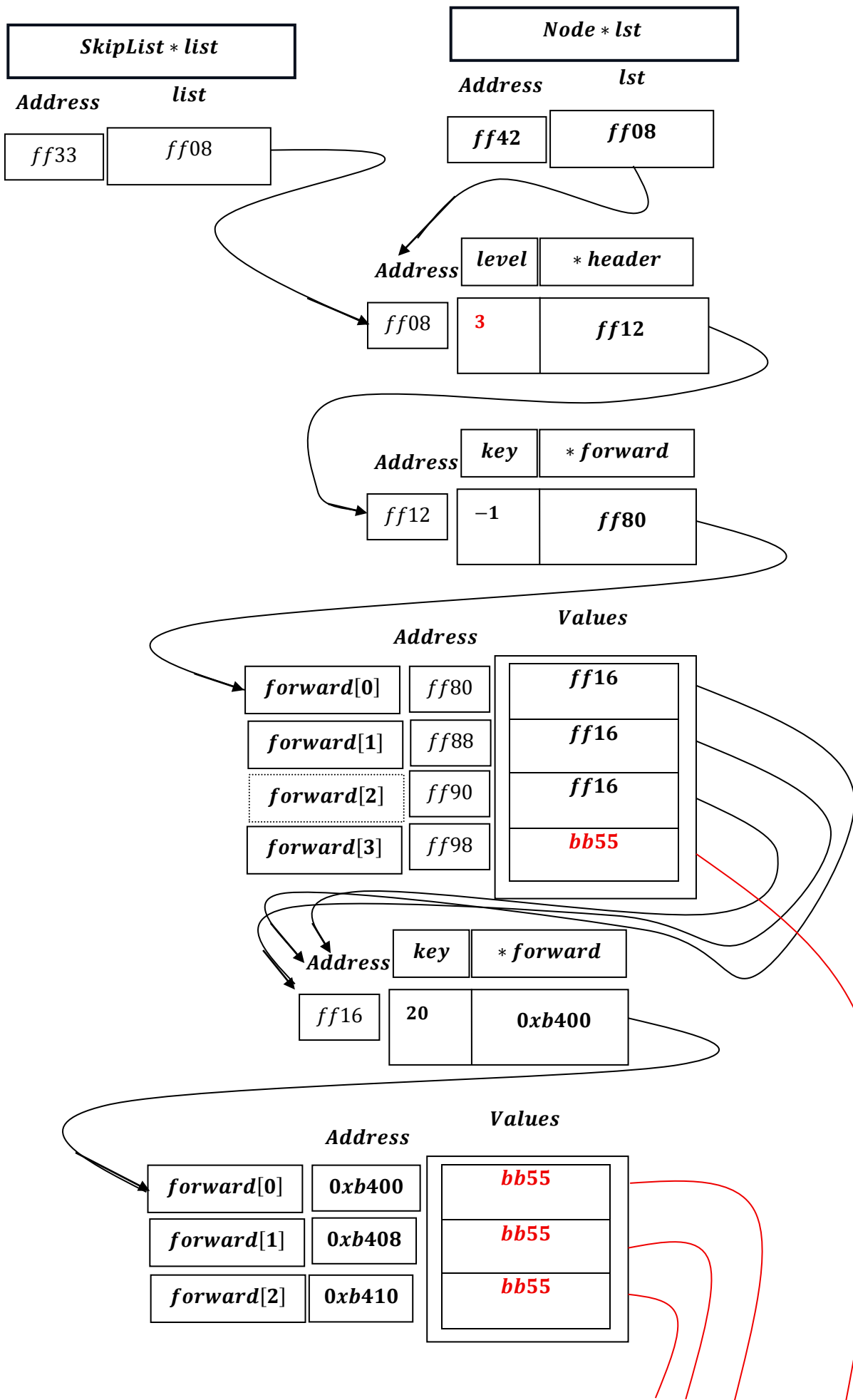
*When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xd800**. Because **0xd808** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

It will not add any offsets and delete each block of memory but all at once.

And ,

```
free(current);
```

Will free the memory block that current pointer pointed to.



*Node * current*

Address

current

ff37

e216

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

NULL

NULL

NULL

NULL

Address

Value

e216

Address

Values

0xd800

0xd808

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55

```

// Reduce the maximum level of the list if layers become empty
while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
    lst->level--;
}
cout << "Successfully deleted key " << key << endl;
}

```

lst → *level* = 3 > 0 [True] AND *lst* → *header: ff12* → *forward*[*lst* → *level: 3*] = *bb55* == *NULL*[False] [**Condition is False**]

Print `` Successfully deleted key : 30``.
